

Laboration 5

Uppgift 1

I uppgift 1 implementerades en generell grafstruktur i Java med hjälp av generics. Grafen består av Vertex-objekt och Edge-objekt som tillsammans representerar noder och kanter i grafen.

Vertex-klassen innehåller information om varje nod, dess koordinater och färg. Edge-klassen representerar en riktad kant mellan två vertices och beräknar avståndet mellan dem med hjälp av euklidiskt avstånd enligt Pythagoras sats.

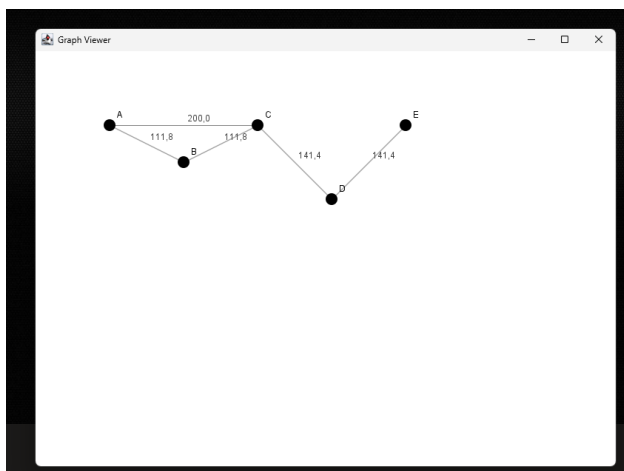
Avståndet mellan två vertices beräknas med hjälp av euklidiskt avstånd enligt formeln:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

där x och y representerar koordinaterna för de två vertices.

Graph-klassen implementerades med hjälp av två HashMaps. Den ena används för att lagra alla vertices medan den andra används för att lagra edges kopplade till varje vertex. För att representera en oriktad kant används två riktade Edge-objekt, ett i varje riktning.

Ett enkelt GUI användes för att visualisera grafen och dess noder och kanter. Samtliga publika metoder i Graph-klassen testades med hjälp av JUnit-tester för att säkerställa att funktionaliteten fungerade korrekt. AI användes som stöd för att förstå implementationen av grafstrukturer, JUnit-tester och generics i Java. All kod analyserades, modifierades och testades manuellt i Eclipse.



Uppgift 2

Dijkstras algoritm används för att hitta den kortaste vägen mellan två noder i en graf. Algoritmen börjar med att ge startnoden avståndet 0 medan alla andra noder får ett väldigt stort värde. Sedan väljs den nod som har det kortaste avståndet och dess grannar undersöks. Om en kortare väg hittas uppdateras avståndet. Algoritmen fortsätter på samma sätt tills den kortaste vägen till målnoden har hittats.

Jag har tidigare arbetat med Dijkstras algoritm i GIS Raster-kursen där vi använde tabeller för att hålla reda på processade noder, kostnad och tidigare nod. Det gjorde det lättare att förstå hur algoritmen arbetar steg för steg.

En bra implementation för Dijkstras algoritm kan bygga på klasserna Graph, Vertex och Edge som användes i uppgiften. Graph-klassen kan använda HashMaps för att lagra alla vertices och edges eftersom det gör det enkelt och snabbt att hitta rätt nod och dess kopplingar.

Jag skulle kunna lägga till några extra attribut i klasserna. I Vertex skulle man kunna lagra det kortaste avståndet och vilken tidigare nod som användes för att nå den. I Edge skulle man kunna lagra en vikt eller kostnad för kanten. I Graph skulle man kunna använda en PriorityQueue för att enklare välja nästa nod med lägst kostnad.

Jag skulle också kunna lägga till metoder för att hitta den kortaste vägen och för att visualisera resultatet i GUI:t.

Typen T skulle till exempel kunna innehålla namn på städer, ID eller annan information som identifierar en vertex. Attributet info behövs eftersom det används för att skilja vertices från varandra.

För att visualisera algoritmens resultat skulle man kunna färga noder och kanter som tillhör den kortaste vägen eller skriva ut vägen och den totala kostnaden i GUI:t eller i konsolen.